

EXAMEN DE FIN DE MODULE

Module : ERP Odoo

Nom : Adama Seck

Niveau : Master 1 Systèmes d'Information

Question 1 — La signification de db_user dans le fichier de configuration Odoo (2 pts)

Réponse correcte : Le nom de l'administrateur de la base de données PostgreSQL.

Odoo utilise PostgreSQL comme système de gestion de base de données. Le paramètre **db_user** figurant dans le fichier de configuration **odoo.conf** désigne le nom d'utilisateur PostgreSQL avec lequel Odoo se connecte à la base de données. Il ne s'agit pas d'un utilisateur applicatif Odoo, mais d'un compte PostgreSQL disposant des droits nécessaires pour créer, lire et modifier les bases de données utilisées par l'ERP.

Question 2 — Le rôle de l'ORM Odoo (2 pts)

L'ORM (Object-Relational Mapping) d'Odoo est une couche d'abstraction qui permet aux développeurs d'interagir avec la base de données PostgreSQL en utilisant des objets Python, sans avoir à écrire directement des requêtes SQL. Chaque modèle Odoo est une classe Python qui hérite de **models.Model**, et chaque instance de cette classe correspond à un enregistrement dans la base de données. L'ORM prend en charge la création des tables, la gestion des relations entre modèles (One2many, Many2one, Many2many), les contraintes d'intégrité, ainsi que les opérations CRUD (Create, Read, Update, Delete). Il simplifie considérablement le développement en garantissant une indépendance par rapport aux spécificités de la base de données sous-jacente.

Question 3 — La différence entre une Vue et une Action dans Odoo (2 pts)

Dans Odoo, une **Vue (View)** définit la manière dont les données d'un modèle sont présentées à l'utilisateur. Il en existe plusieurs types : la vue formulaire (form), la vue liste (tree/list), la vue kanban, la vue graphique, etc. Chaque vue est associée à un modèle et décrit la structure de l'interface affichée.

Une **Action (Action)** quant à elle, définit ce qui se passe lorsqu'un utilisateur navigue vers un menu ou clique sur un bouton. L'action la plus courante est la **ir.actions.act_window** qui lie un modèle à une ou plusieurs vues et détermine le contexte d'affichage (domaine de filtrage, vue par défaut, etc.). En résumé, la Vue répond à la question « comment afficher les données ? » tandis que l'Action répond à « que faire et quoi montrer ? ».

Question 4 — La signification de l'attribut _inherits dans un fichier modèle (2 pts)

Réponse correcte : De mapper le nom (_name) de l'objet.

L'attribut **_inherits** permet à un modèle Odoo de déléguer certains champs à un autre modèle existant via une relation Many2one. Contrairement à **_inherit** qui étend directement un modèle, **_inherits** crée un nouveau modèle qui « pointe » vers un autre et accède à ses champs comme s'ils lui appartenaient. Cela revient à mapper l'identité (_name) du nouveau modèle sur un enregistrement du modèle parent, sans duplication de table.

Question 5 — L'attribut model pour créer une vue dans la balise record (2 pts)

Dans la balise **<record>** d'un fichier XML de données Odoo, l'attribut permettant de créer une vue est :

```
model="ir.ui.view"
```

C'est via ce modèle technique qu'Odoo enregistre toutes les vues (formulaires, listes, kanbans, etc.) dans la base de données. Le champ **arch** de cet enregistrement contient la structure XML de la vue.

Question 6 — L'attribut model pour créer une action dans la balise record (2 pts)

Pour créer une action de fenêtre (la plus courante), on utilise :

```
model="ir.actions.act_window"
```

Ce modèle est utilisé pour définir les actions qui ouvrent une ou plusieurs vues sur un modèle donné. Il existe également d'autres types d'actions comme **ir.actions.server** (actions serveur) ou **ir.actions.report** (rapports), selon le besoin.

Question 7 — L'architecture 3 tiers de l'ERP Odoo (2 pts)

L'architecture d'Odoo repose sur trois couches distinctes qui communiquent entre elles :

1. La couche présentation (Tier 1) : Il s'agit de l'interface utilisateur, accessible depuis un navigateur web. Odoo propose un client web basé sur JavaScript (framework OWL — Odoo Web Library) qui communique avec le serveur via des requêtes JSON-RPC.

2. La couche applicative (Tier 2) : C'est le cœur du système, constitué du serveur Odoo développé en Python. Il contient la logique métier, l'ORM, les modules fonctionnels (comptabilité, ventes, stocks, etc.) et gère les communications entre l'interface et la base de données.

3. La couche données (Tier 3) : Elle est assurée par PostgreSQL, le SGBD relationnel utilisé par Odoo pour stocker l'ensemble des données de l'application. Toutes les interactions avec cette couche passent obligatoirement par l'ORM.

Question 8 — Explication de la ligne : `from . import models` (2 pts)

Cette ligne Python se trouve généralement dans le fichier `__init__.py` d'un module Odoo. Elle indique à Python d'importer le sous-module (ou fichier) nommé **models** depuis le package courant (représenté par le point « . »).

Concrètement, cela permet de charger le fichier **models.py** (ou le sous-dossier **models/** contenant un `__init__.py`) situé dans le même répertoire que le fichier courant. Dans le contexte Odoo, c'est grâce à cette instruction que le serveur prend connaissance des modèles définis dans le module et les enregistre correctement au démarrage.

Question 9 — Procédure pour rediriger un serveur Odoo sur le port 80 (2 pts)

Par défaut, Odoo écoute sur le port 8069. Pour le rendre accessible sur le port 80 (port HTTP standard), il est recommandé d'utiliser un reverse proxy, car faire écouter Odoo directement sur le port 80 nécessiterait des droits root, ce qui est déconseillé en production.

Procédure avec Nginx (recommandée) :

1. Installer Nginx sur le serveur : `sudo apt install nginx`
2. Créer un fichier de configuration dans `/etc/nginx/sites-available/odoo` avec une directive `proxy_pass http://127.0.0.1:8069`; dans un bloc `location /` pour le port 80.
3. Activer la configuration : `sudo ln -s /etc/nginx/sites-available/odoo /etc/nginx/sites-enabled/`
4. Redémarrer Nginx : `sudo systemctl restart nginx`

Cette configuration permet à Nginx de recevoir les requêtes sur le port 80 et de les transmettre au serveur Odoo sur le port 8069, de manière transparente pour l'utilisateur.

Question 10 — Création d'un nouveau répertoire de module dans Odoo (2 pts)

Pour créer un nouveau module Odoo, il faut respecter une structure minimale standardisée. Voici la procédure :

1. **Créer le répertoire du module** dans le chemin défini par `addons_path` dans le fichier `odoo.conf` : par exemple `/odoo/addons/mon_module/`.
2. **Créer le fichier `__manifest__.py`** : ce fichier décrit le module (nom, version, auteur, dépendances, fichiers de données, etc.).
3. **Créer le fichier `__init__.py`** : nécessaire pour que Python reconnaisse le répertoire comme un package. Il contient en général les imports des sous-modules (`from . import models`).
4. **Créer les sous-dossiers nécessaires** : typiquement `models/`, `views/`, `security/`, `data/`.
5. **Mettre à jour la liste des modules** depuis l'interface Odoo (menu Paramètres > Mettre à jour la liste des applications) ou via la commande : `python odoo-bin -u mon_module -d ma_base`.

Une fois ces étapes réalisées, le module apparaît dans la liste des applications installables d'Odoo.